

Bazele Inteligenței Artificiale

Lab 3. Modele neparametrice

Efrem Dragoș-Sebastian-Mihaly

Tipuri de modele în ML

- În general, sunt două tipuri mari și late:

Supervizate

Nesupervizate

Tipuri de modele în ML

- În general, sunt două tipuri mari și late:

Clasificare

utilizate pentru a clasifica datele în categorii distincte

Exemple: regresia logistică, regresia softmax

Regresie

utilizate pentru a prezice valori numerice continue

Exemple: regresia liniară, regresia polinomială, regresia ridge, regresia lasso, etc.

Atenție! Regresia logistică și regresia softmax sunt tot modele de clasificare, chiar dacă conțin cuvântul regresie.

Clasificare

- Haideți să ne reamintim setul de date Iris:

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
5.1	3.5	1.4	0.2	0
7.0	3.2	4.7	1.4	1
6.3	3.3	6.0	2.5	2
...	...	...	...	...

Caracteristici / Attribute / Features

Etichetă / Clasă

Clasificare

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
5.1	3.5	1.4	0.2	0
7.0	3.2	4.7	1.4	1
6.3	3.3	6.0	2.5	2
...	...	...	...	...

Caracteristici / Attribute / Features

Etichetă / Clasă

0 - Iris-Setosa

1 - Iris-Versicolor

2 - Iris-Virginica

Clasificare

Sepal Length	Sepal Width	Species
5.1	3.5	0
7.0	3.2	1
6.3	3.3	2
...	...	...

Caracteristici / Attribute / Features

Etichetă / Clasă

0 - Iris-Setosa

1 - Iris-Versicolor

2 - Iris-Virginica

Clasificare

K-Nearest Neighbors

K-Nearest Neighbors

Sepal Length	Sepal Width	Species
5.1	3.5	0
7.0	3.2	1
6.3	3.3	2
...	...	...

Caracteristici / Attribute / Features

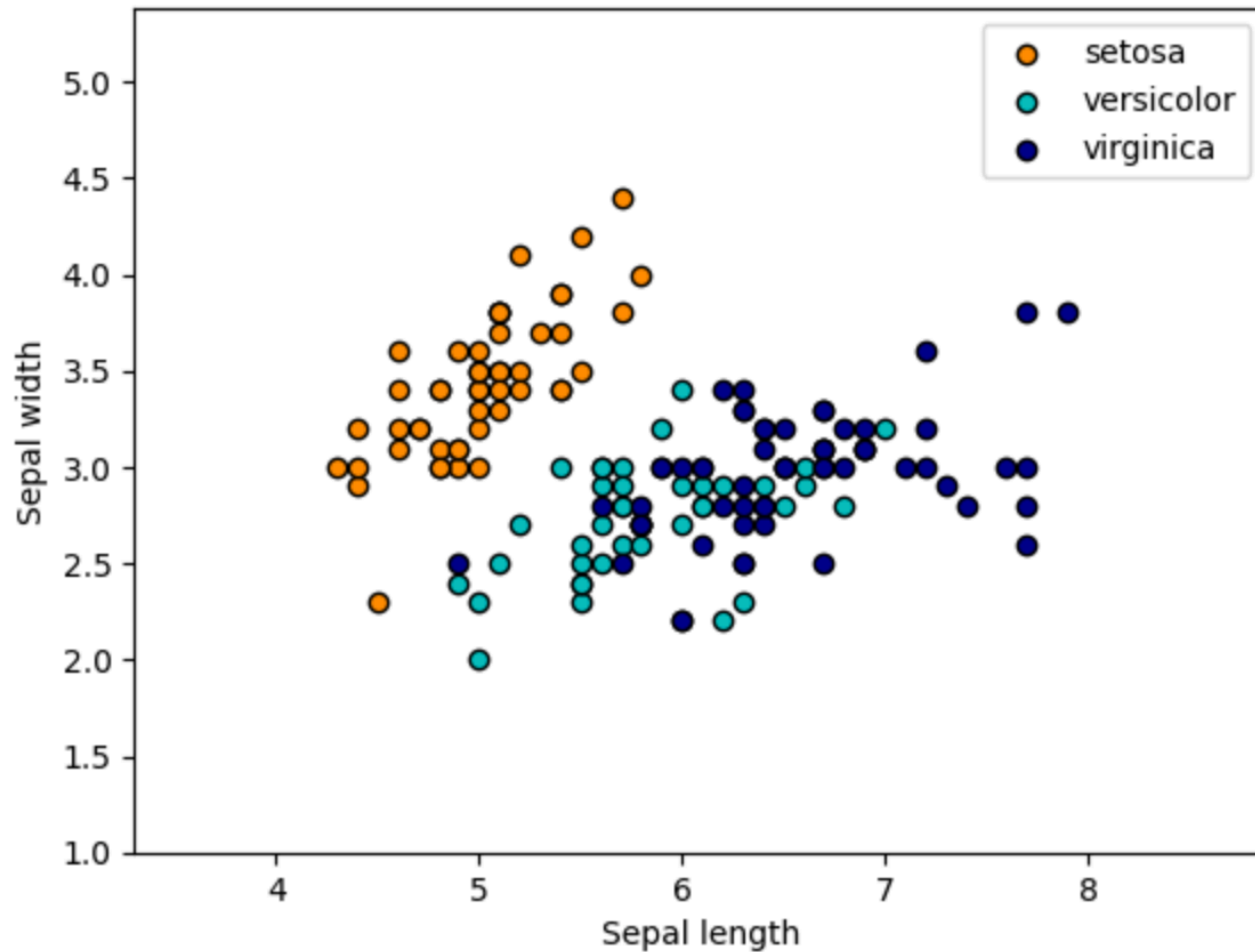
Etichetă / Clasă

0 - Iris-Setosa

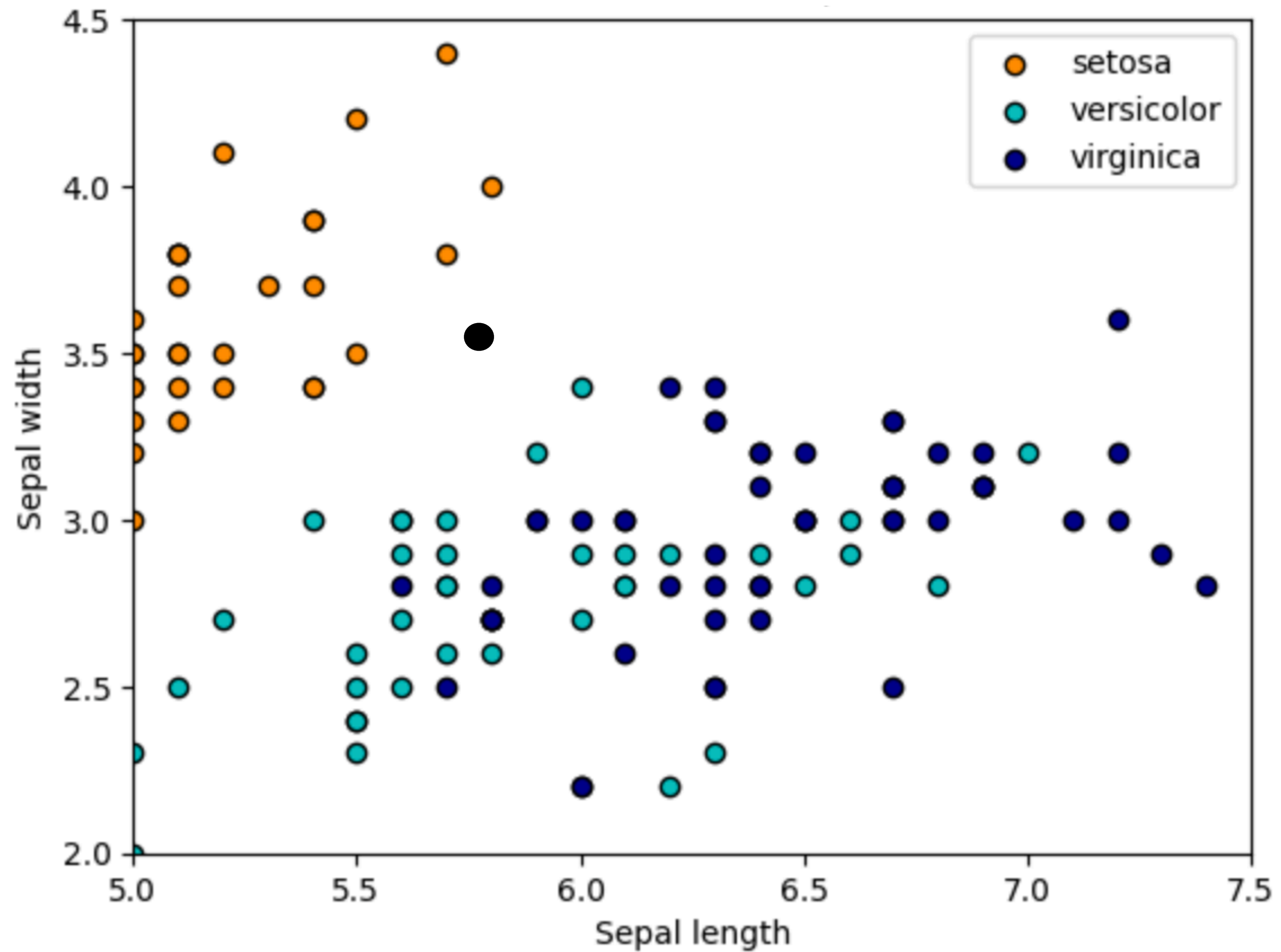
1 - Iris-Versicolor

2 - Iris-Virginica

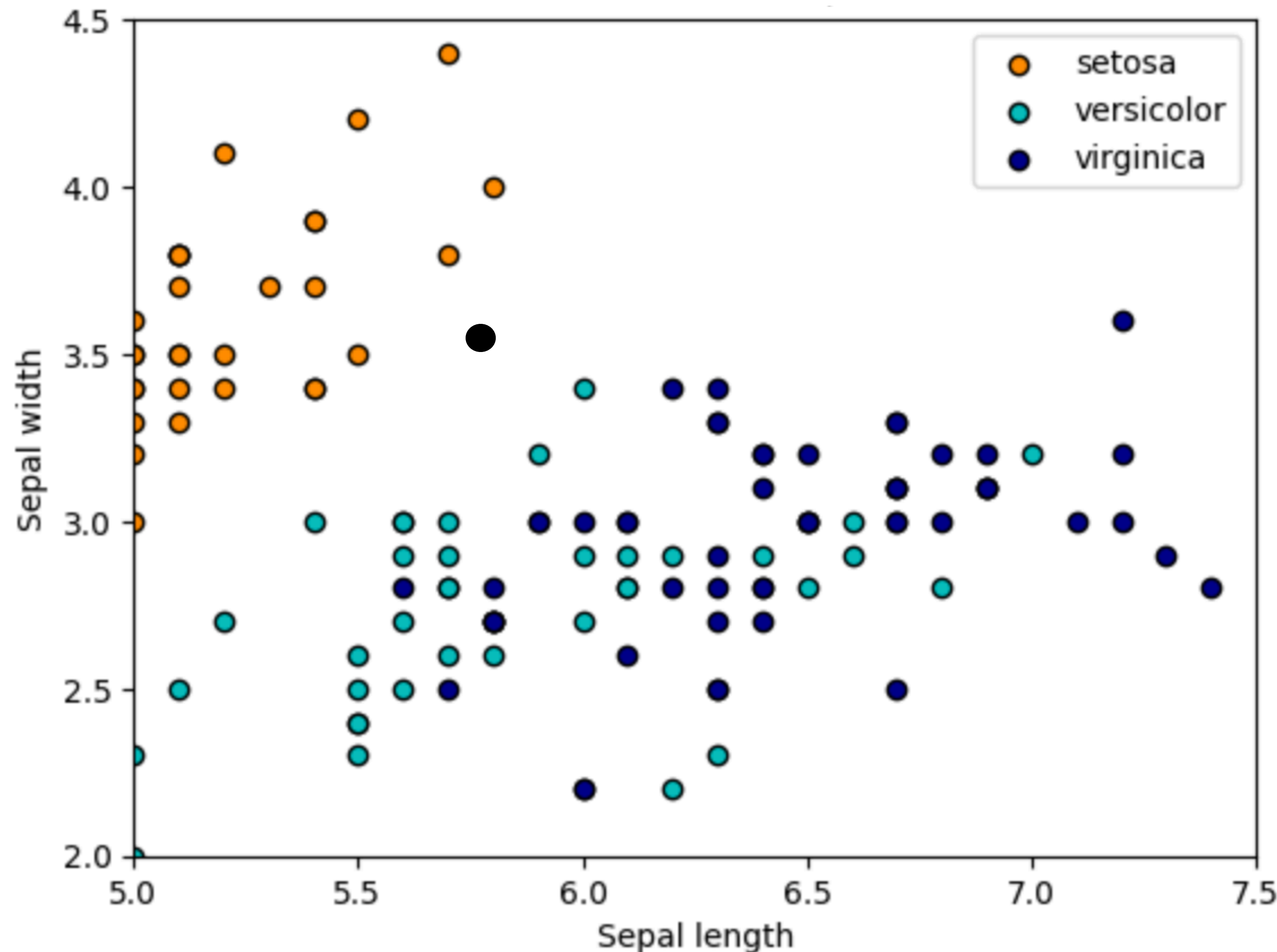
K-Nearest Neighbors



K-Nearest Neighbors

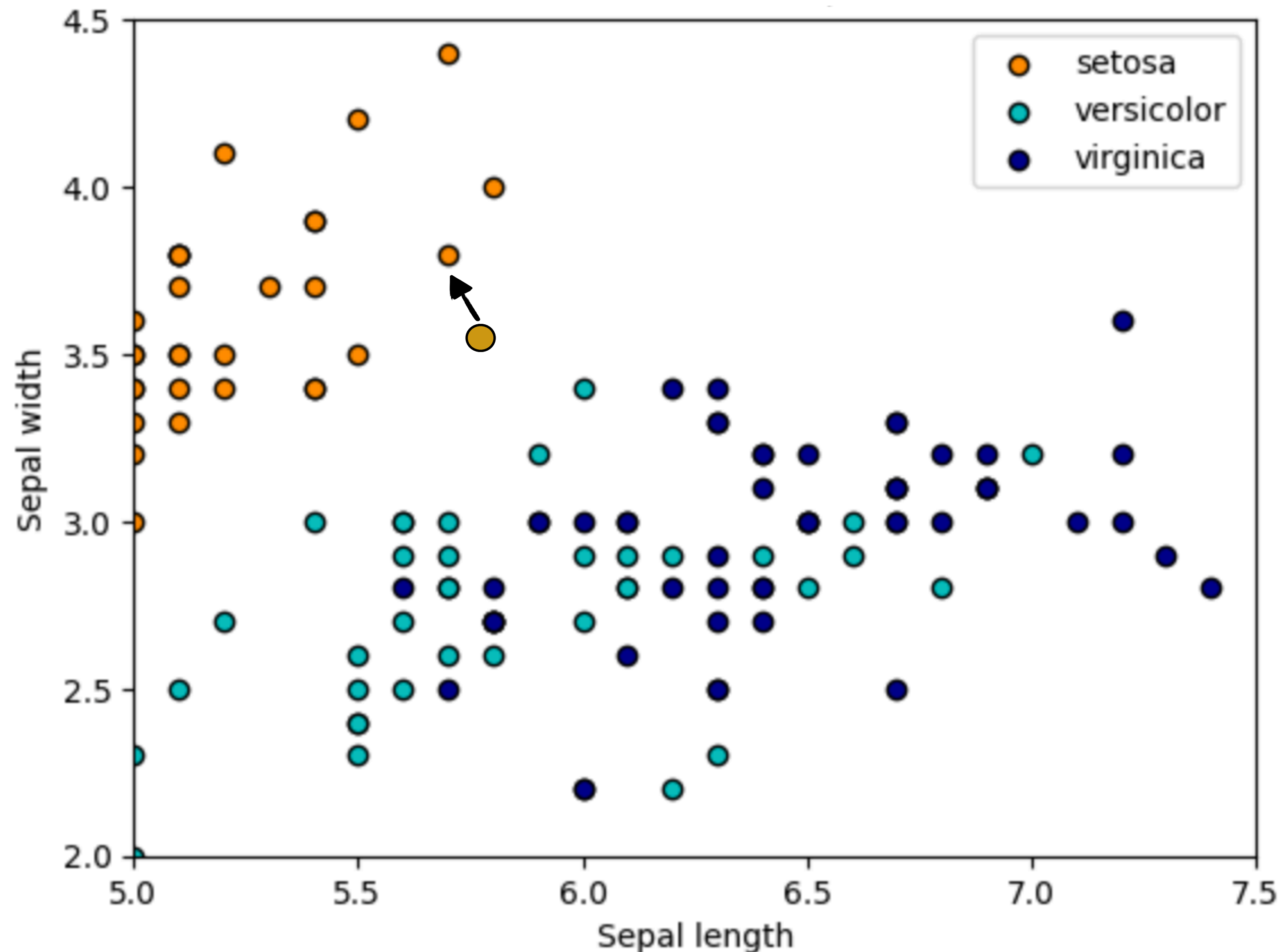


K-Nearest Neighbors



- un algoritm simplu și intuitiv de clasificare care funcționează pe baza similarității între puncte de date.
- când avem un exemplu nou de floare (un punct nou de date), KNN caută în setul de date cele mai apropiate k flori, calculând distanța dintre floarea nouă și toate florile din setul de antrenament. În general, se folosește distanța Euclidiană pentru a măsura această apropiere.

K-Nearest Neighbors



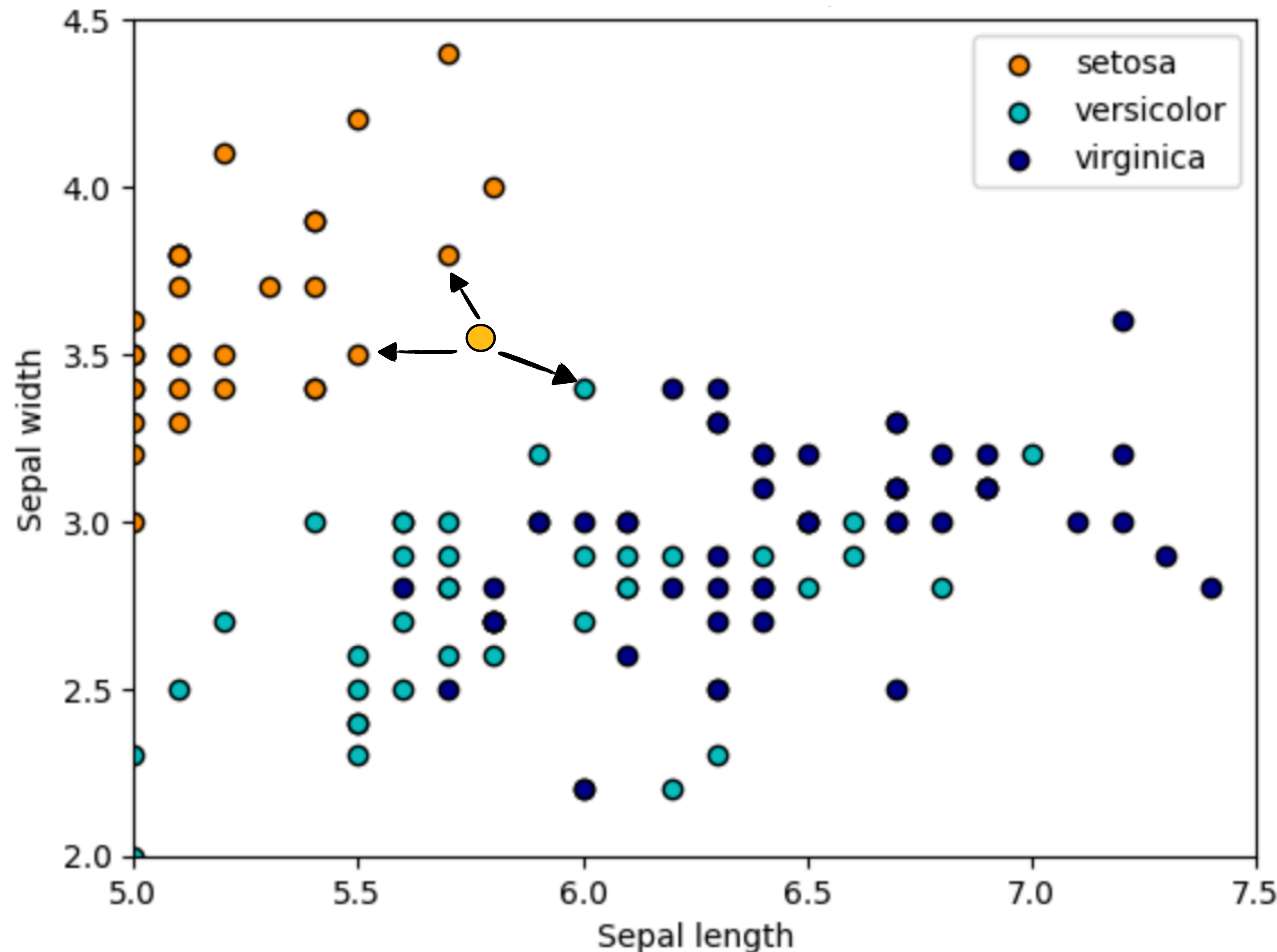
Exemplu:

La $k = 1$:

Algoritmul va căuta floarea care este cea mai apropiată de punctul nou pe baza distanței Euclidiene.

Noua floare va fi clasificată exact ca specia vecinului său cel mai apropiat.

K-Nearest Neighbors



Exemplu:

La $k = 3$:

Algoritmul va căuta cele mai apropiate 3 flori față de punctul nou pe baza distanței Euclidiene.

Floarea nouă va fi clasificată pe baza speciei majoritare dintre cei trei vecini. Asta înseamnă că fiecare dintre cei trei vecini “votează” pentru specia lor, iar specia care are cele mai multe voturi va fi eticheta finală.

K-Nearest Neighbors

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Incarcam setul de date Iris
iris_dataset = load_iris() # 3 clase, 4 features
X, y = iris_dataset.data, iris_dataset.target

# Impartim setul de date astfel: 75% in training, 25% in test

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

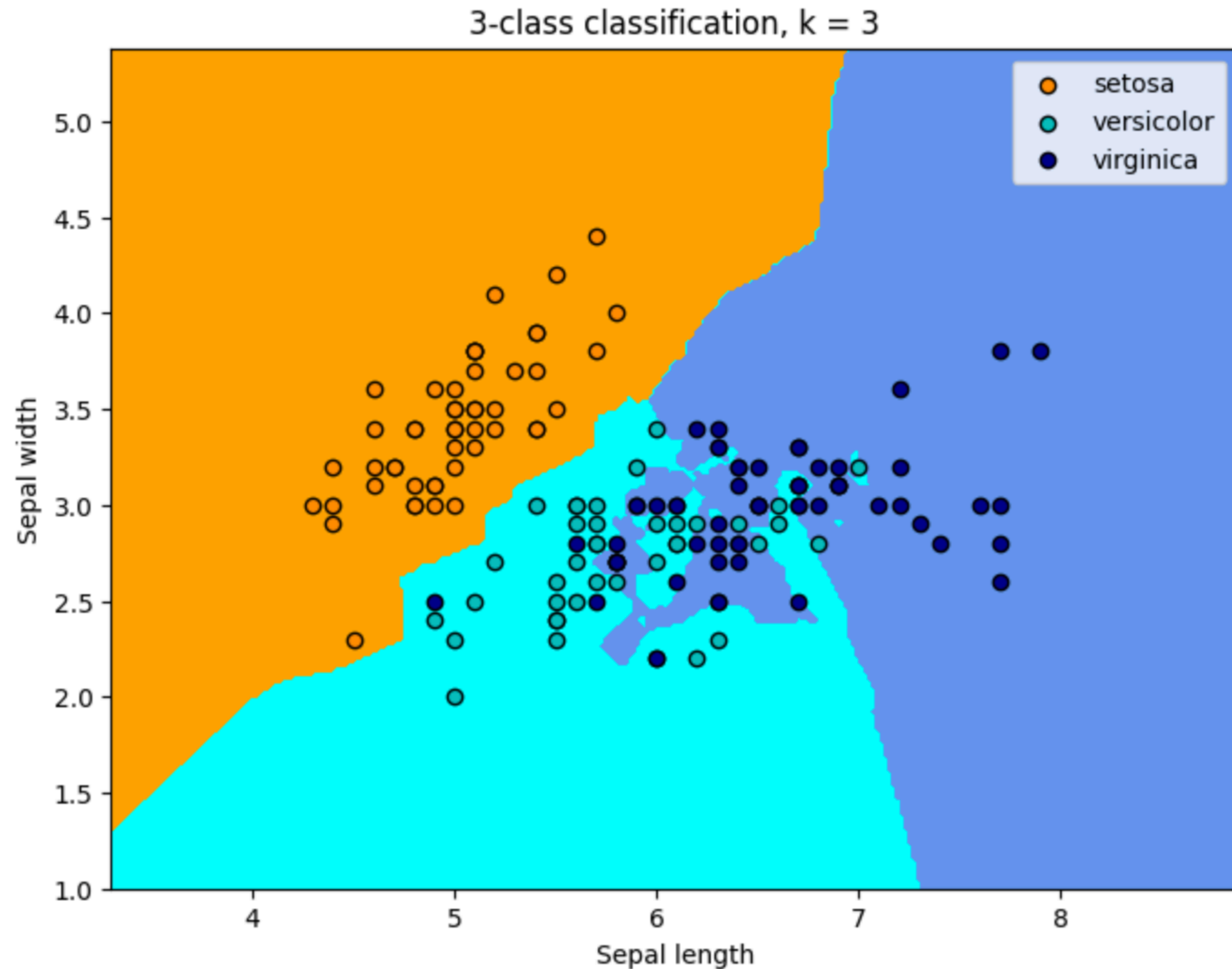
# Initializam K Nearest Neighbors cu K = 3
clf = KNeighborsClassifier(n_neighbors=3)
clf.fit(X_train, y_train)

# Afisam: etichetele de test (ground-truth), etichetele de test (prezise) si acuratetea pe test

y_test, clf.predict(X_test), clf.score(X_test, y_test)
```

(array([1, 0, 2, 1, 1, 0, 1, 2, 1, 1, 2, 0, 0, 0, 0, 1, 2, 1, 1, 2, 0, 2,
0, 2, 2, 2, 2, 2, 0, 0, 0, 0, 1, 0, 0, 2, 1, 0]),
array([1, 0, 2, 1, 1, 0, 1, 2, 1, 1, 2, 0, 0, 0, 0, 1, 2, 1, 1, 2, 0, 2,
0, 2, 2, 2, 2, 2, 0, 0, 0, 0, 1, 0, 0, 2, 1, 0]),
1.0)

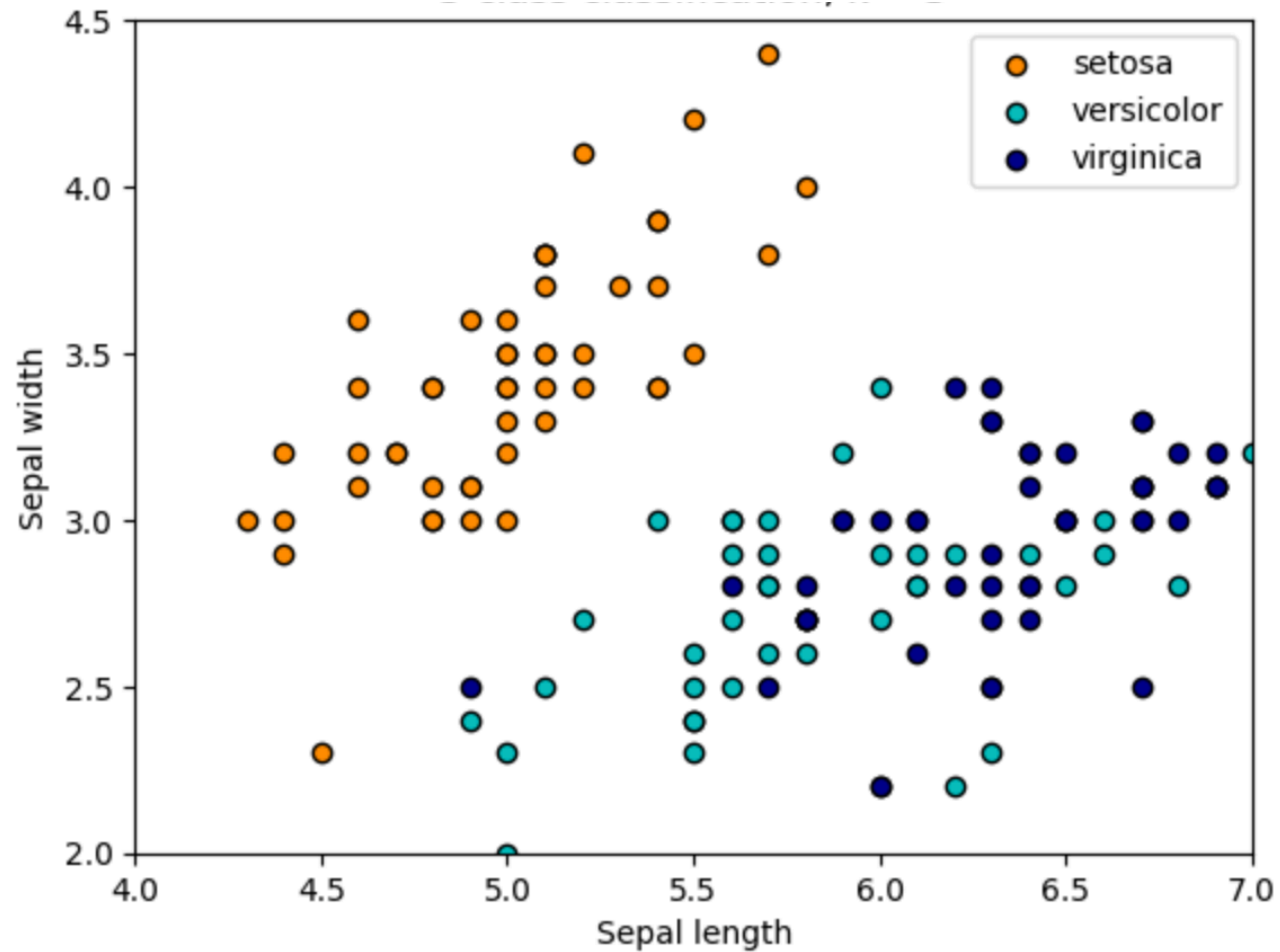
K-Nearest Neighbors



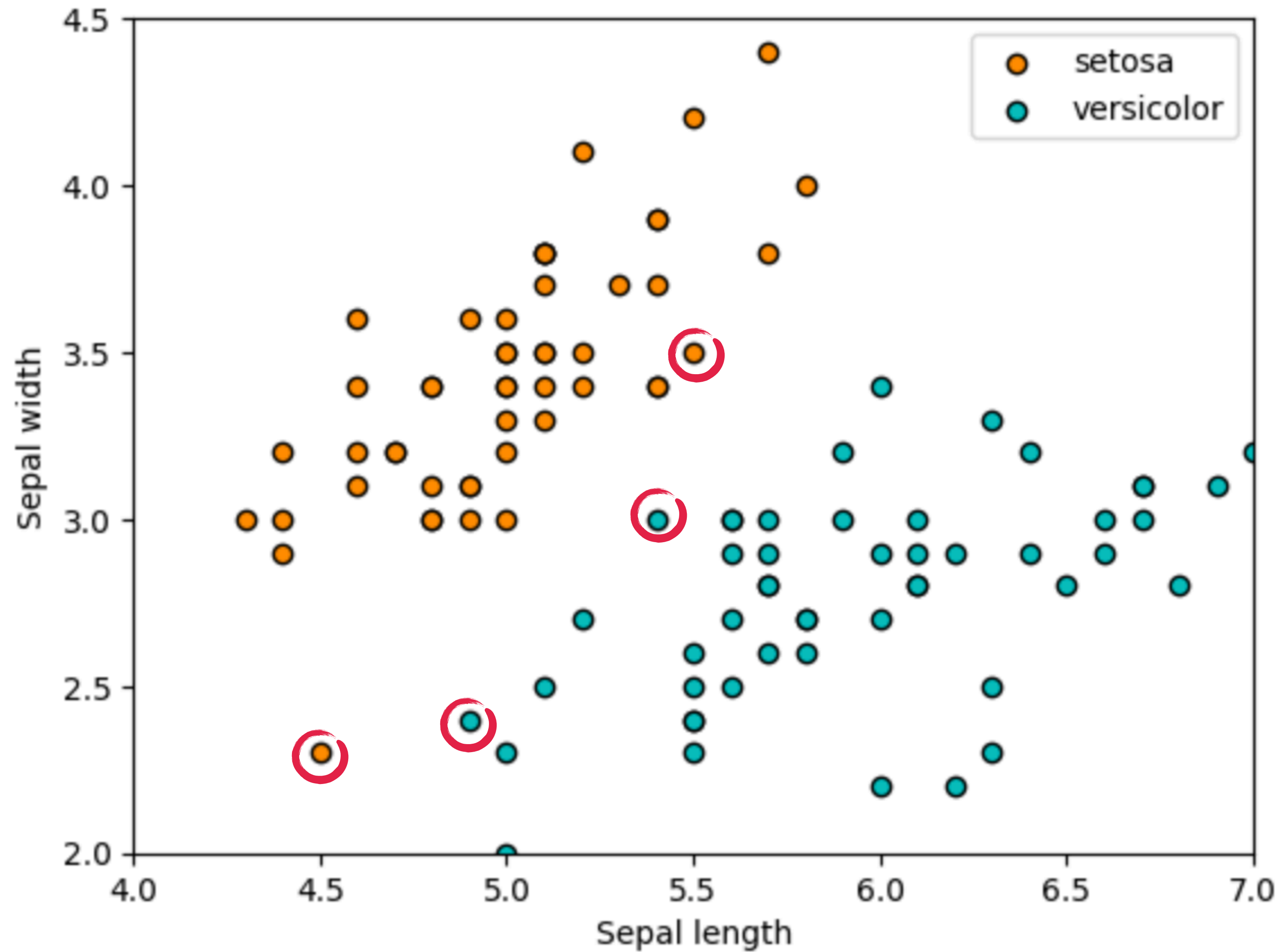
K-Nearest Neighbors

- Mai poartă și numele de model non-parametric
- Spre deosebire de modelele parametrice, cum ar fi regresia liniară, care au un set fix de parametri ce definesc modelul, KNN memorează toate datele de antrenament și face predicții pe baza vecinilor celor mai apropiați de un punct nou.
- Practic, spunem că e model non-parametric pentru că nu face nicio presupunere explicită despre distribuția datelor sau despre o funcție fixă care leagă variabilele de intrare și ieșire.

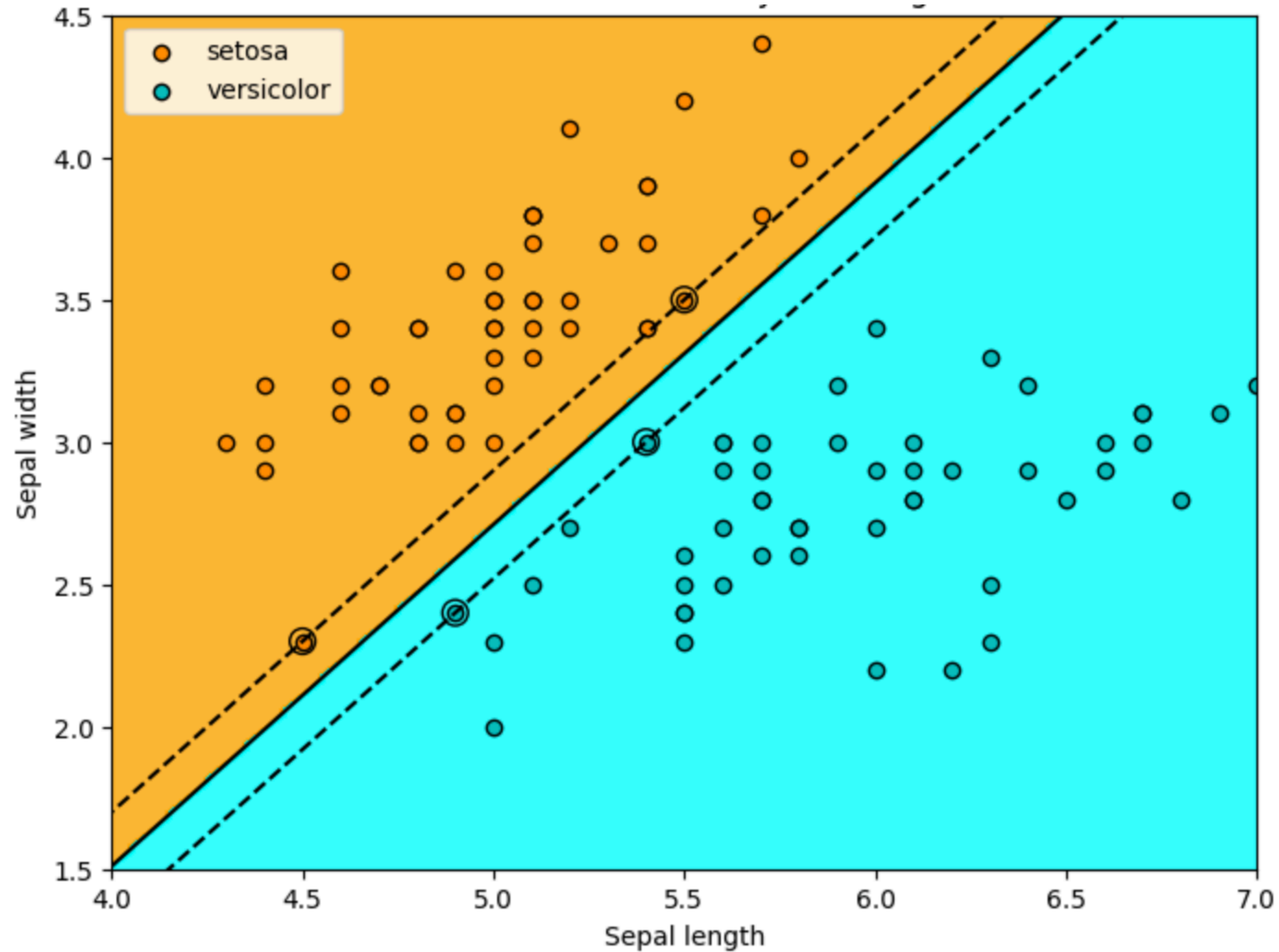
Clasificare



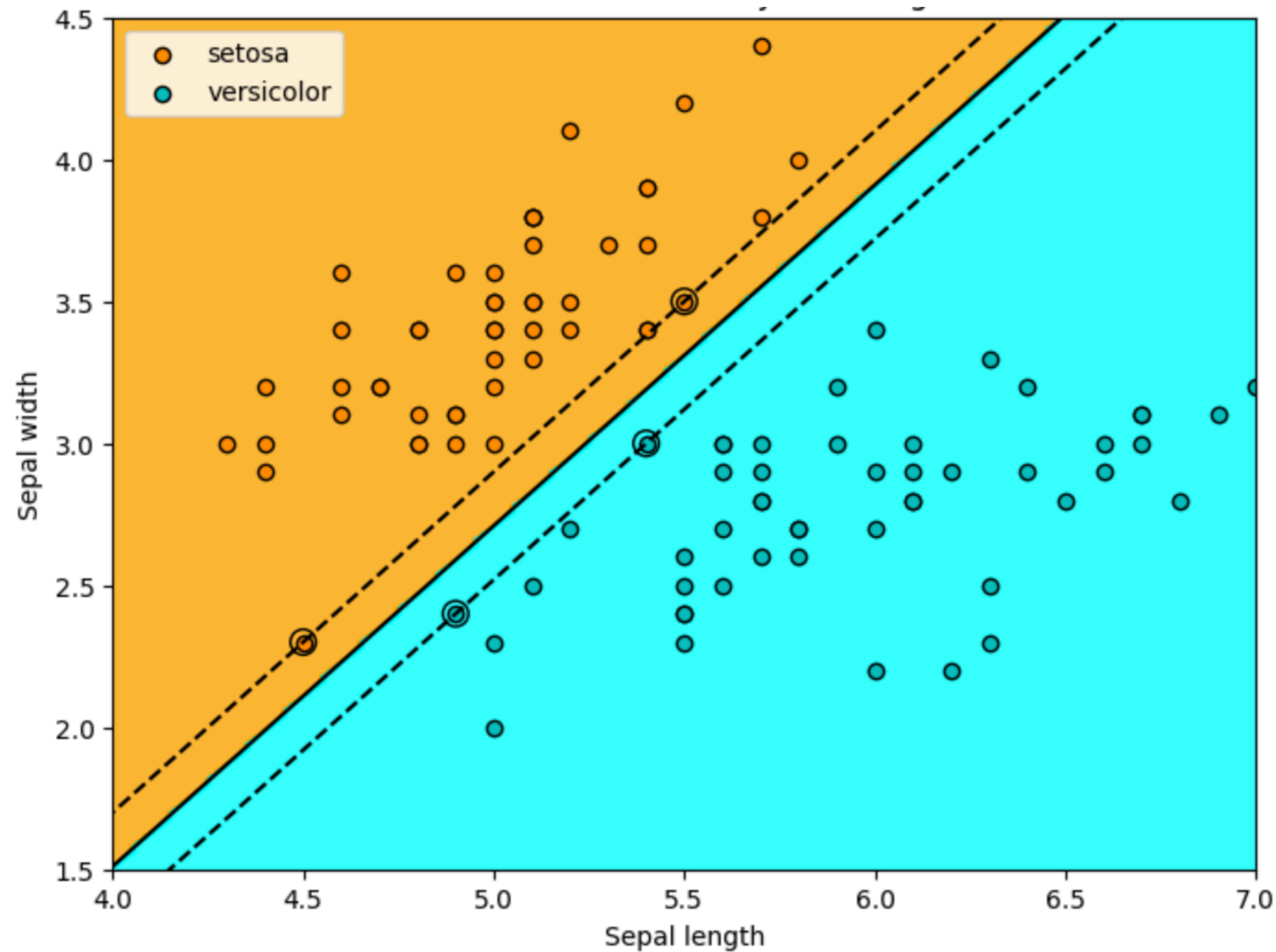
Clasificare



Hard-Margin SVM

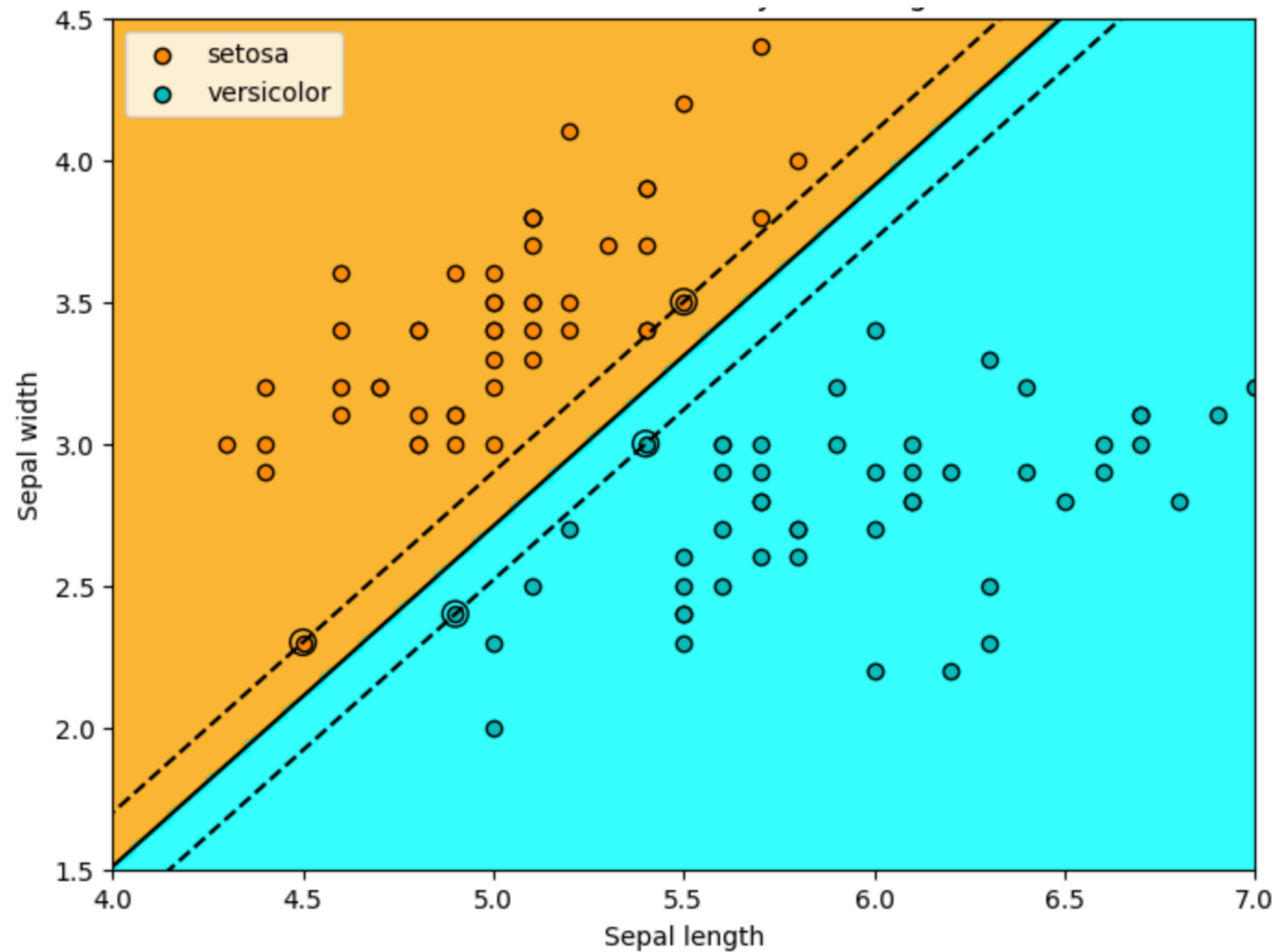


Hard-Margin SVM



- SVM încearcă să găsească un hyperplan optim care separă clasele de date, astfel încât să maximizeze marginea (distanța) dintre punctele de date ale diferitelor clase.
- SVM caută să găsească nu doar orice linie de separare, ci cea care maximizează distanța dintre cele mai apropiate puncte ale claselor (numite support vectors). Cu cât aceste puncte sunt mai departe de linia de separare, cu atât modelul este considerat mai bun.

Hard-Margin SVM



- După ce a fost găsit hyperplanul optim, noi puncte de date (flori) pot fi clasificate pe baza poziției lor față de acest hyperplan.
- Dacă se află de partea stângă a liniei, va fi clasificat ca Iris-setosa. Dacă se află pe partea dreaptă a liniei, va fi clasificat ca Iris-versicolor.

Hard-Margin SVM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

iris_dataset = load_iris()

# Consideram doar primele 2 caracteristici (sepal width & height)
X = iris_dataset.data[:, :2]
y = iris_dataset.target
valid = (y == 0) | (y == 1) # In exemplul asta, vom face clasificare binara intre Iris-Setosa si Iris-Versicolor
X = X[valid]
y = y[valid]

# Split in training set si test set

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

clf = SVC(kernel='linear', C=float('inf'))
clf.fit(X_train, y_train)

clf.coef_, clf.intercept_, clf.support_vectors_, clf.score(X_test, y_test)
```

⇒ (array([[6.3154899 , -5.26238666]]),
array([-17.31642492]),
array([[5.5, 3.5],
[4.5, 2.3],
[4.9, 2.4],
[5.4, 3.]]),
1.0)

Hard-Margin SVM

```
clf = SVC(kernel='linear', C=float('inf'))
```

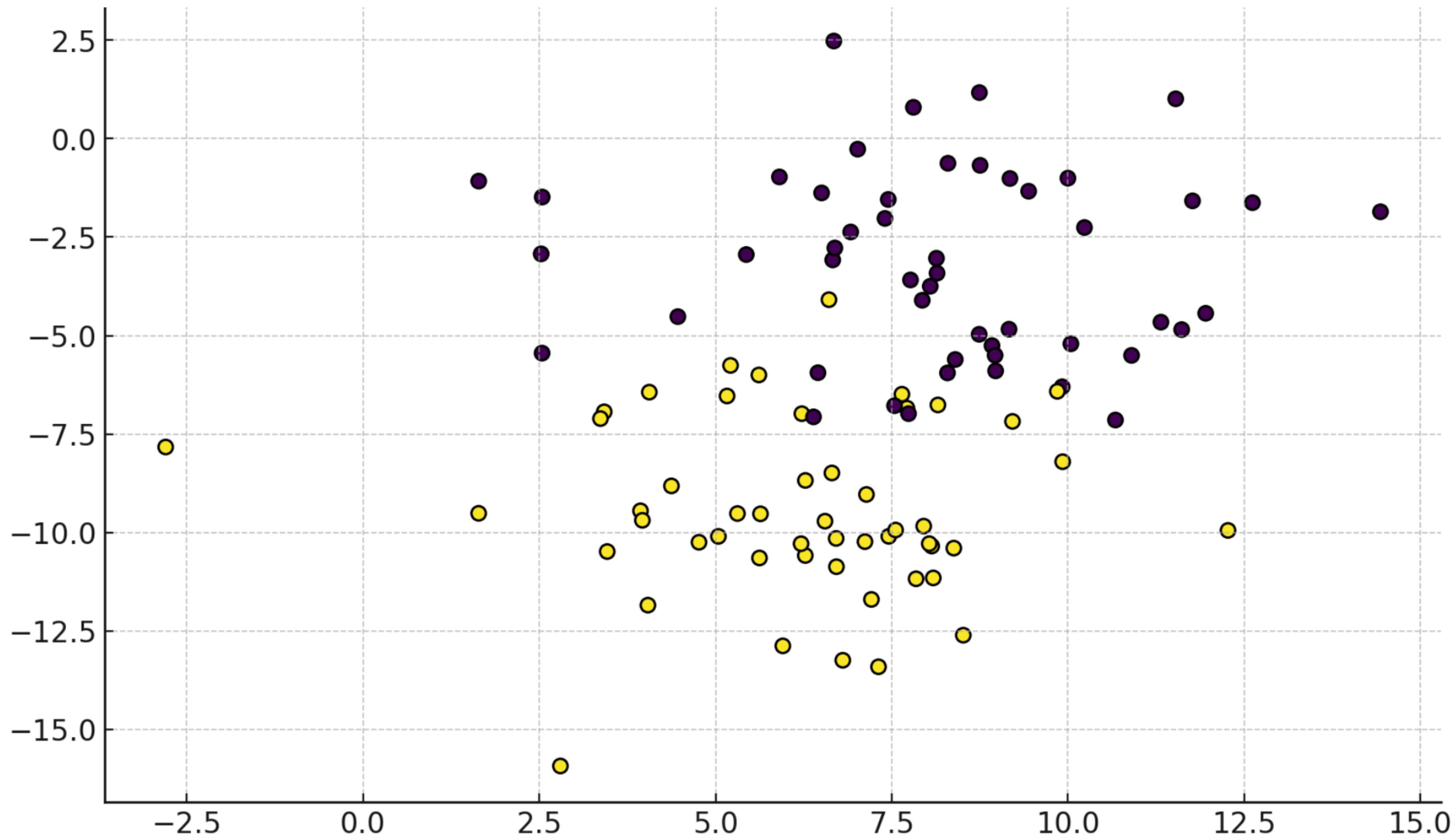
Obligatoriu pentru separare liniară

Trebuie să fie infinit pentru hard-margin SVM

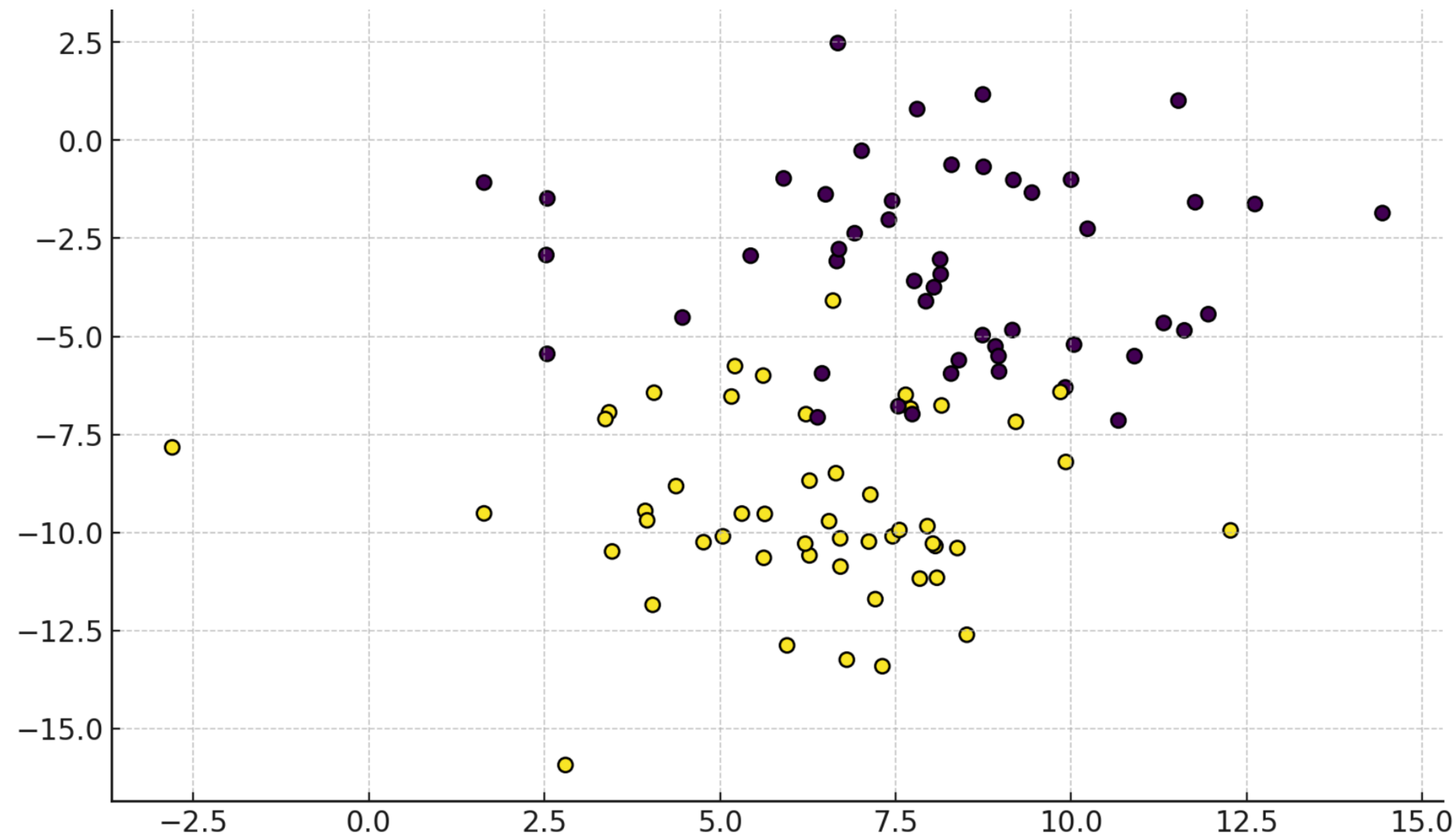
Hard-Margin SVM: Folosit când datele sunt perfect separabile liniar.

Dar dacă datele nu sunt perfect separabile liniar?

Dar dacă datele nu sunt perfect separabile liniar?



Dar dacă datele nu sunt perfect separabile liniar?



Hard  **Soft**

Găsirea unei linii good enough

Soft-Margin SVM

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm
from sklearn.datasets import make_blobs

# Un dataset cu putin overlap intre cele doua clase
X, y = make_blobs(n_samples=100, centers=2, random_state=6, cluster_std=2.5)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Create SVM soft-margin
clf = SVC(kernel='linear', C=0.5)

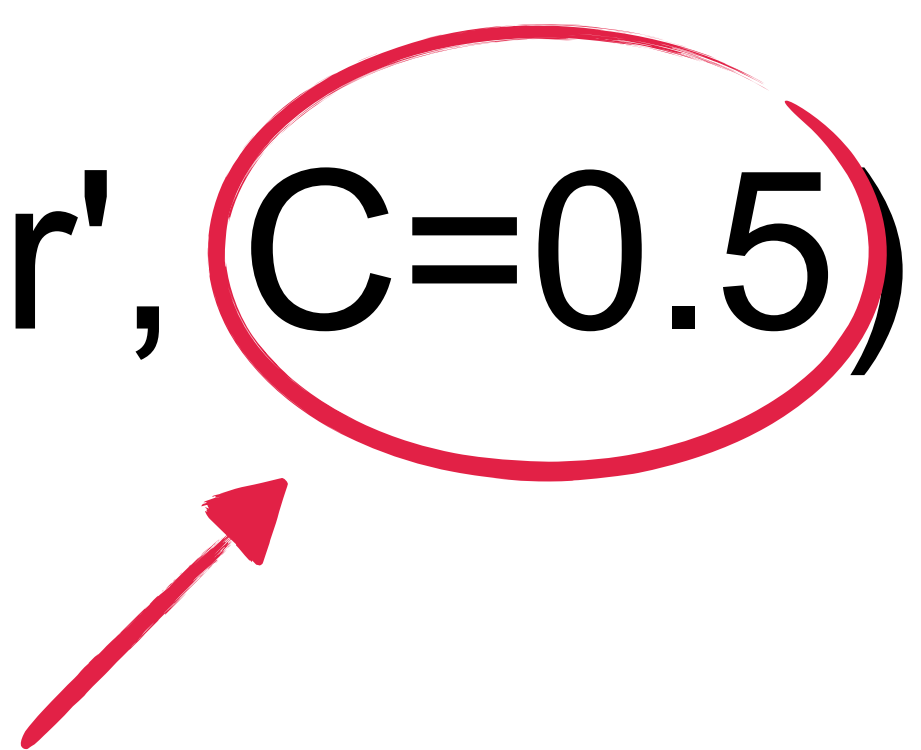
# Antrenare model
clf.fit(X_train, y_train)

clf.coef_, clf.intercept_, clf.score(X_test, y_test)
```

(array([[-0.29243291, -0.60888361]]), array([-1.46909247]), 0.88)

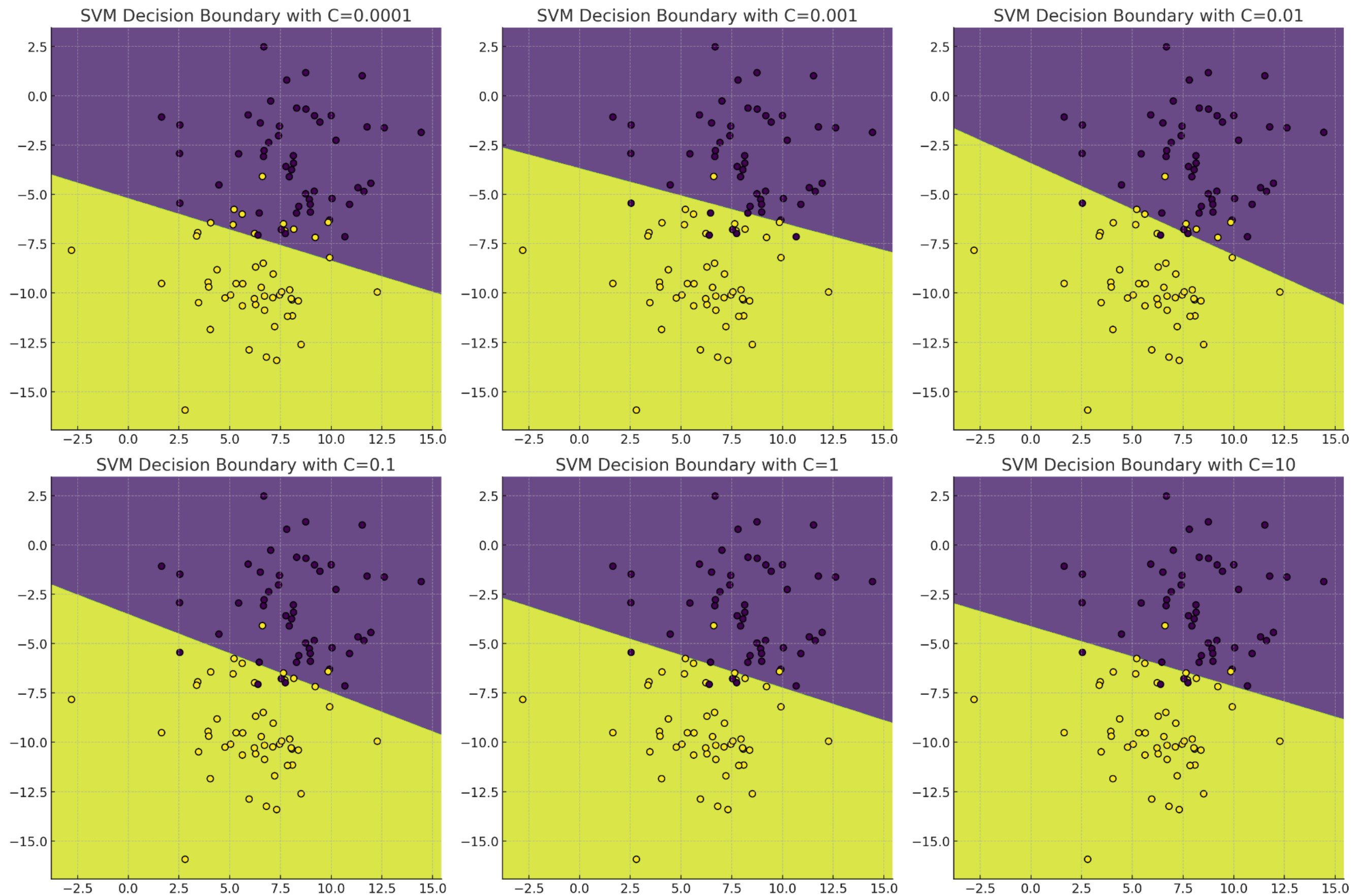
Soft-Margin SVM

```
clf = SVC(kernel='linear', C=0.5)
```



Cât timp C nu e infinit, clasa SVC va reprezenta Soft-Margin SVM

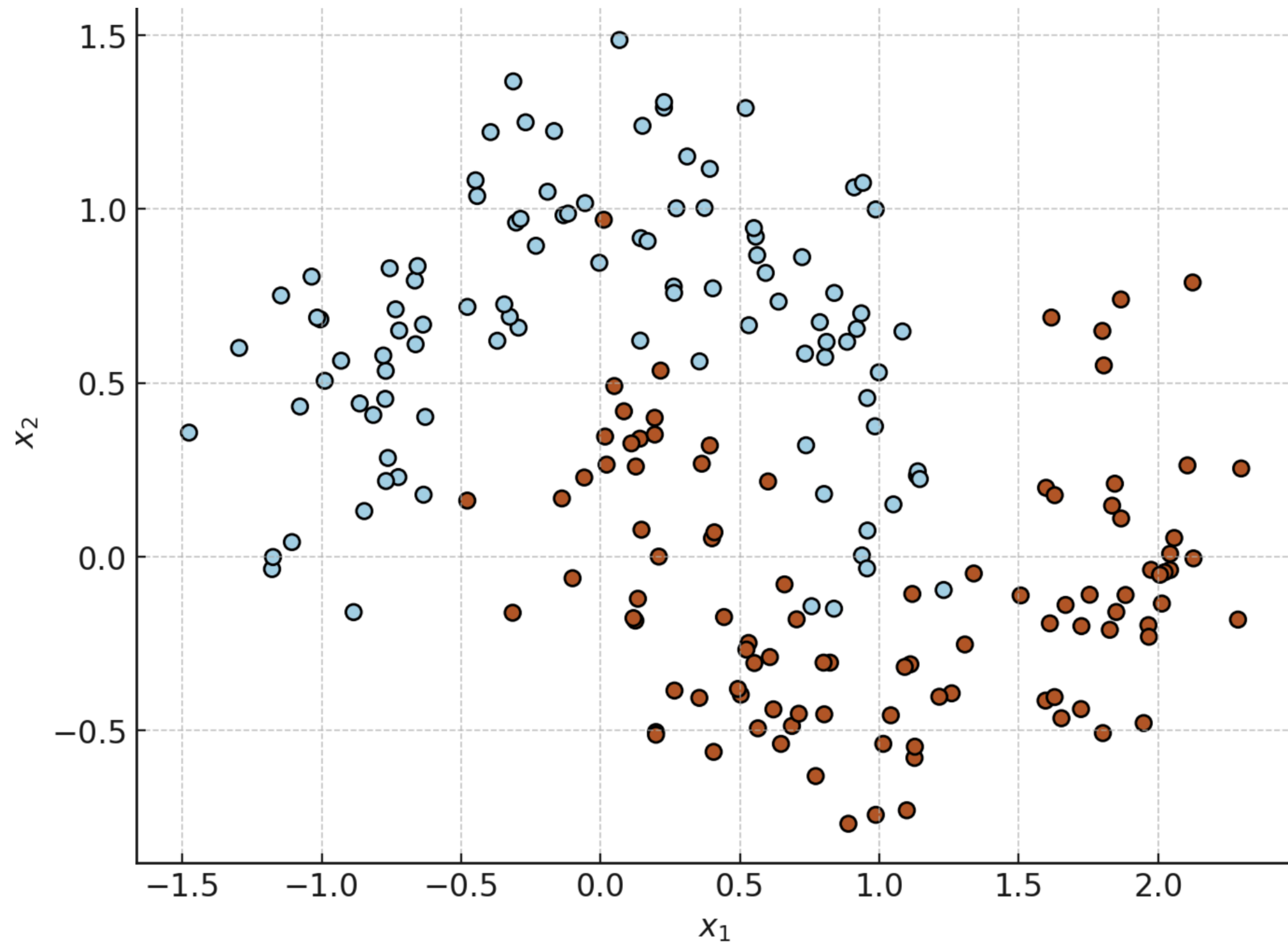
Soft-Margin SVM



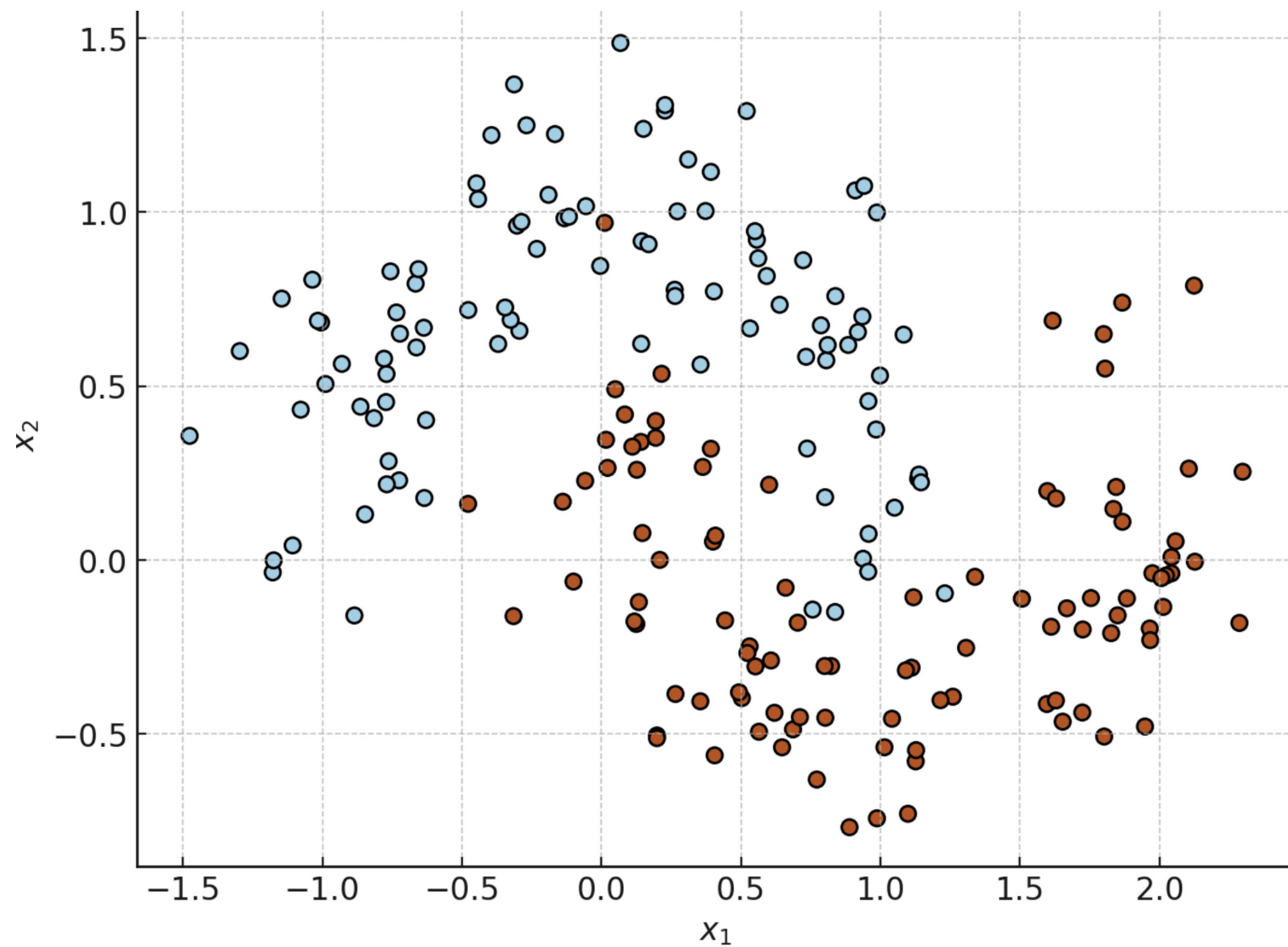
Cu un C mic, modelul devine mai permisiv și permite ca mai multe puncte să cadă pe partea greșită a marginii.

Cu un C mare, modelul penalizează mai mult greșelile, așa că va încerca să corecteze fiecare eroare chiar dacă asta înseamnă o margine mai îngustă.

SVM Non-Liniare

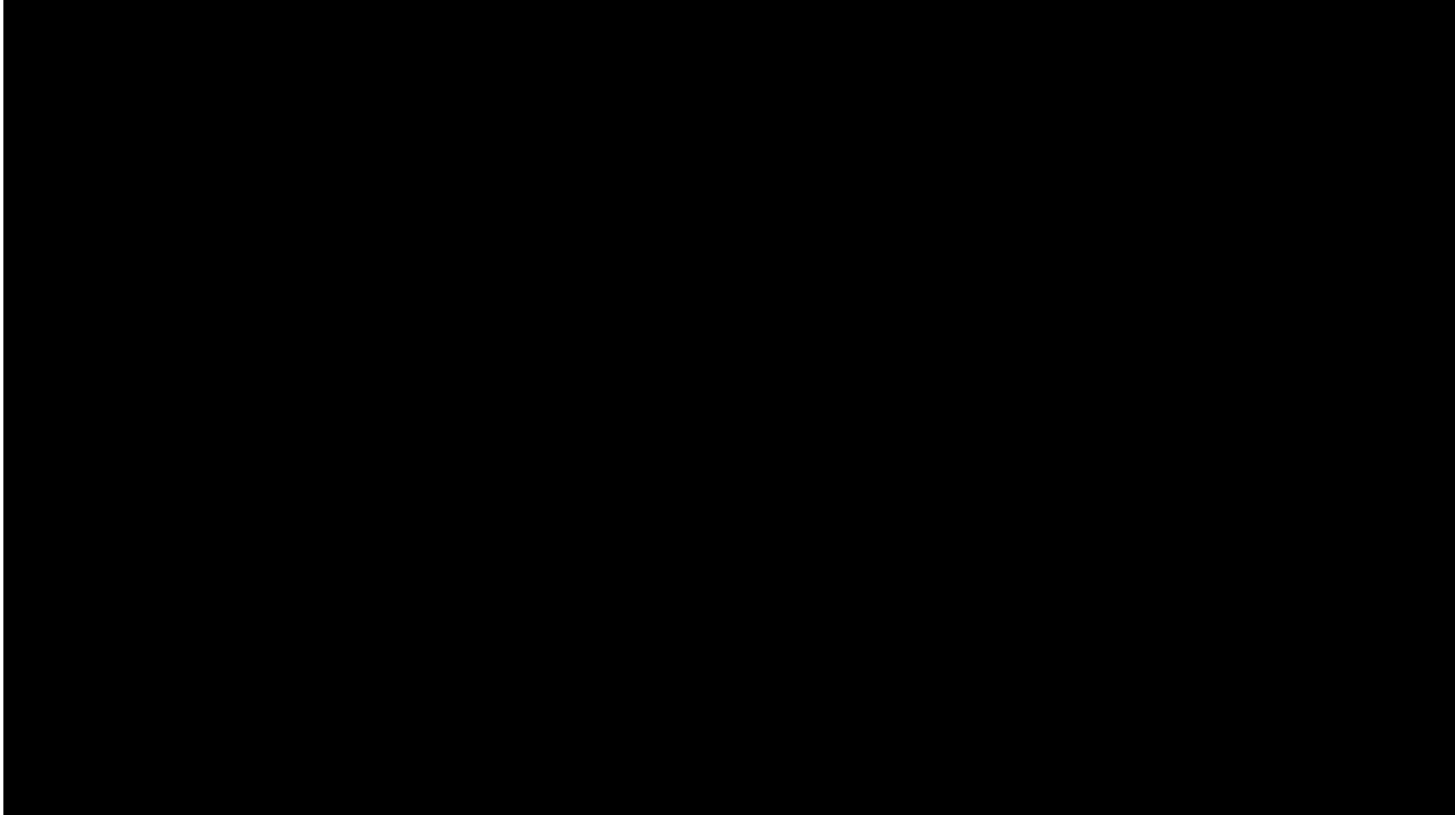


SVM Non-Liniare



SVM poate funcționa și pe date non-liniare dacă acestea sunt transpuse într-un spațiu mai mare.

SVM Non-Linear



SVM Non-Liniare

```
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Folosim setul de date Moons
X, y = make_moons(n_samples=200, noise=0.2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Creare pipeline scalare + SVM
svm_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm_clf', SVC(kernel='poly', degree=3, coef0=1, C=1.0))
])

# Antrenare model
svm_pipeline.fit(X_train, y_train)

# Prezicere pe test
y_pred = svm_pipeline.predict(X_test)

# Acuratete pe test
accuracy_score(y_test, y_pred)
```

0.95

SVM Non-Liniare

```
svm_pipeline = Pipeline([  
    ('scaler', StandardScaler()),  
    ('svm_clf', SVC(kernel='poly', degree=3, coef0=1, C=1.0))  
])
```

Pipeline este folosit atunci când vrem să ~~legăm mai multe acțiuni pe~~ setul nostru de date unul după altul.

Scalarea este necesară când folosim SVM pe date non-liniare.

Kernelul polinomial mapează implicit spațiul de intrare original într-un spațiu de dimensiuni superioare, unde clasele pot fi mai ușor separate.

Scalarea (Și importanța la SVM)

Presupunem că avem două caracteristici:

- numărul de cuvinte dintr-un mail (poate varia de la 10 la 1000)
- numărul de link-uri dintr-un mail (poate varia de la 0 la 5)

Dacă nu scanăm datele, caracteristica „număr de cuvinte” va domina caracteristica „număr de link-uri”

SVM, fiind un algoritm bazat pe distanțe, va putea considera că caracteristica „număr de cuvinte” este mai importantă decât „număr de link-uri” doar pentru că are scala diferită, nu neapărat pentru că are o influență reală asupra clasificării.

Scalarea (Și importanța la SVM)

- numărul de cuvinte dintr-un mail (poate varia de la 10 la 1000)
- numărul de link-uri dintr-un mail (poate varia de la 0 la 5)

Scalarea (StandardScaler() în Scikit-Learn) va face ca toate caracteristicile să fie între 0 și 1, rezolvând această problemă.

SVM Non-Liniare

```
svm_pipeline = Pipeline([  
    ('scaler', StandardScaler()),  
    ('svm_clf', SVC(kernel='poly', degree=3, coef0=1, C=1.0))  
])
```

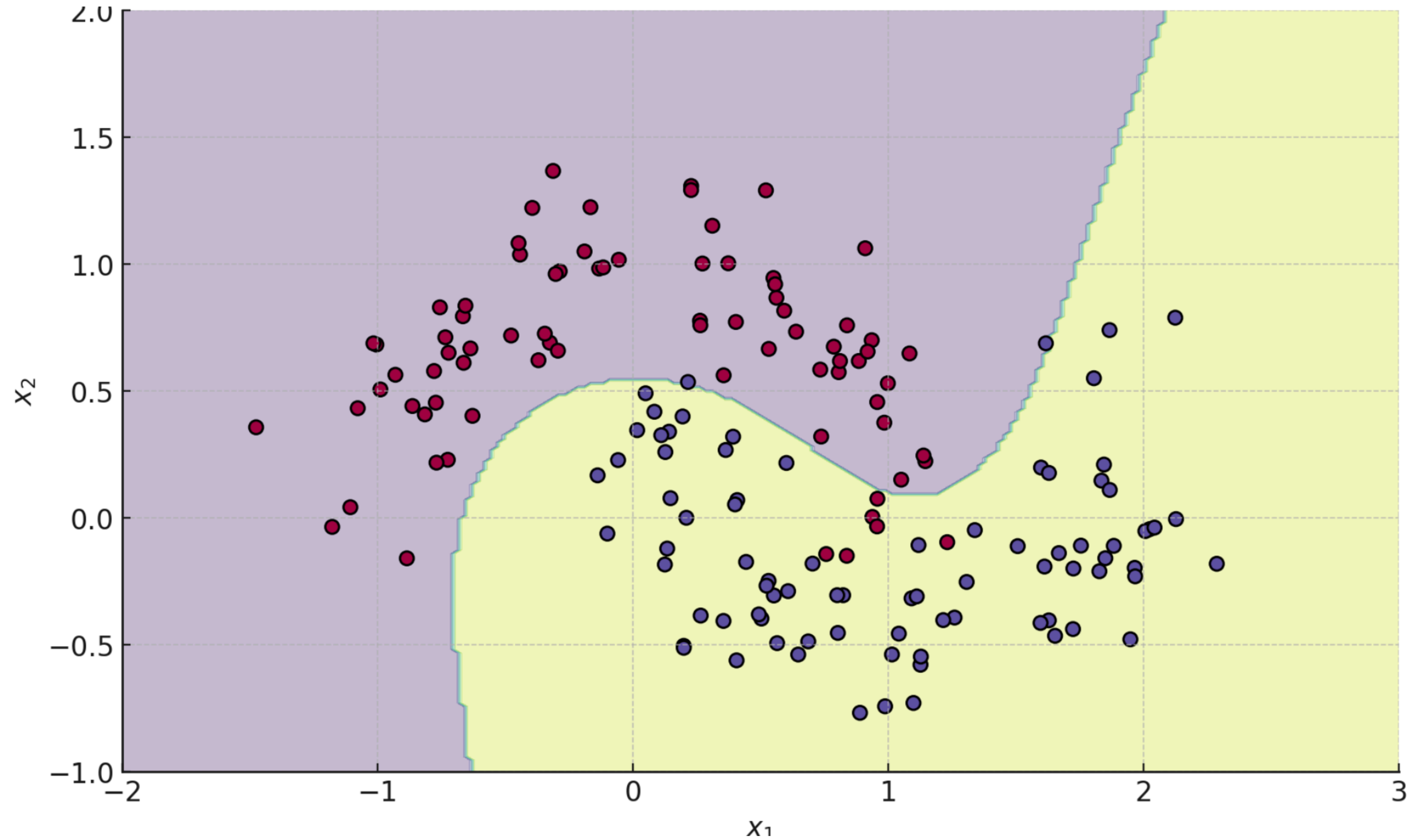
În acest caz, vom folosi svm_pipeline la fel cum am folosit pentru SVM obișnuit, doar că acum va face și scalare înainte să îi dea modelului „să mănânce” datele.

```
clf = SVC(kernel='linear', C=0.5)
```



Aici doar SVM obișnuit, fără scalare la început

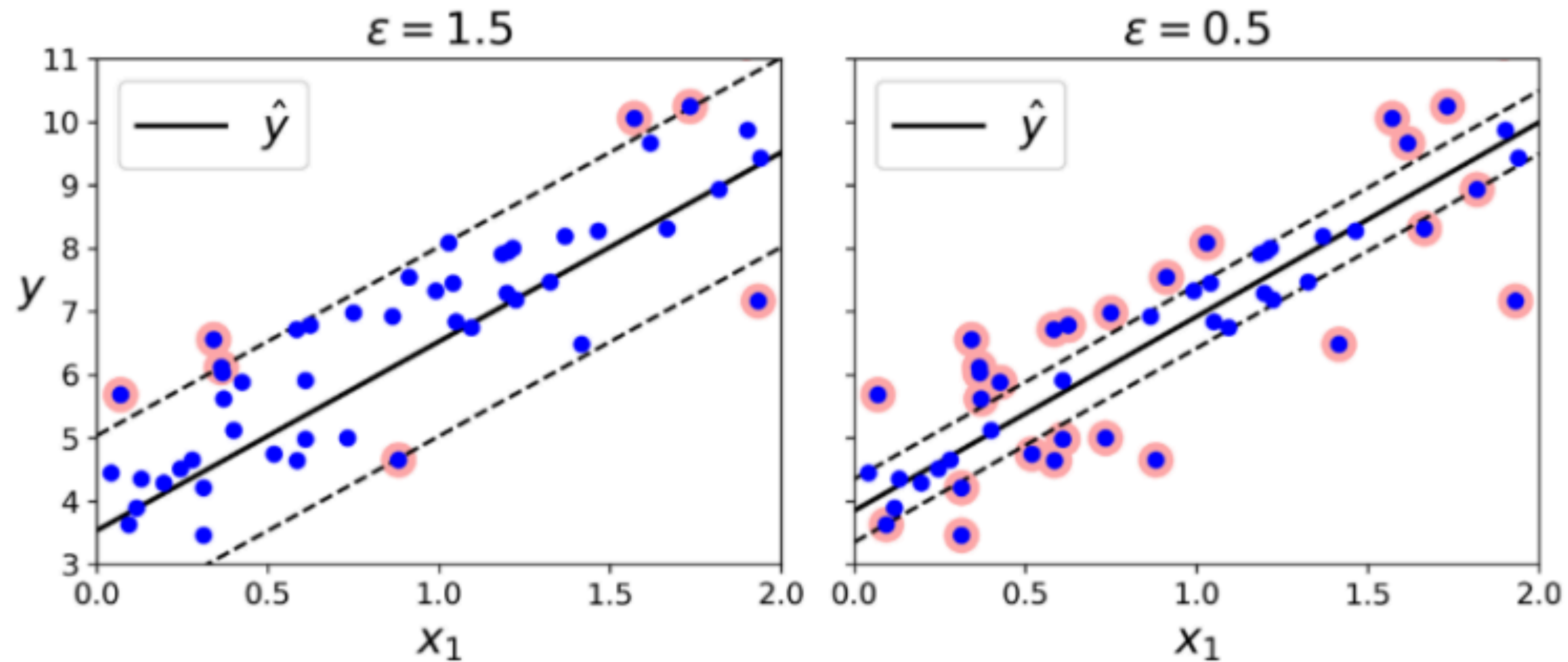
SVM Non-Liniare



Regresia SVM

- Spre deosebire de SVM-ul clasic folosit pentru clasificare, unde separăm clase, în regresie SVM încercăm să prezicem o valoare continuă.
- Regresia liniară caută o linie care minimizează eroarea totală pentru toate punctele.
- Regresia SVM adaugă o bandă de toleranță în jurul liniei de predicție. Această bandă permite ca punctele care se află în interiorul ei să nu fie penalizate (nu contribuie la eroare). ~~Penalizăm doar punctele care sunt în afara benzii, făcând modelul mai flexibil și robust.~~

Regresia SVM



Regresia SVM este ca o regresie liniară, dar adaugă acea bandă de toleranță (ϵ) pentru a face modelul mai flexibil.

Regresia SVM

```
from sklearn.svm import LinearSVR

np.random.seed(42)
n = 50
X = 2 * np.random.rand(n, 1)
y = (4 + 3 * X + np.random.randn(n, 1)).ravel()

svm_reg1 = LinearSVR(epsilon=1.5, random_state=42)
svm_reg1.fit(X, y)
```

Coeficient de toleranță



Regresia SVM non-liniară

```
from sklearn.svm import SVR

n = 100
X = 2 * np.random.rand(n, 1) - 1
y = (0.2 + 0.1 * X + 0.5 * X**2 + np.random.randn(n, 1)/10).ravel()

svm_poly_reg1 = SVR(kernel="poly", degree=2, C=100, epsilon=0.1)
svm_poly_reg1.fit(X, y)
```

Parametru de regularizare

Coeficient de toleranță

C mare = risc de overfitting.

Epsilon mare = model mai simplu, risc mai mic de overfitting, dar și posibil underfitting dacă este prea mare.

That's it for today!
Now let's do some exercises.

That's it for today!
Now let's do some exercises.

Laboratory 3 – Exercises

1. Apply the k-nearest neighbors algorithm on the `moons` dataset. The number of samples of this dataset is 200. Split the data such that the test split is 45% of the initial dataset. Set the number of neighbors parameter to 2 and compute the accuracy on the test set. Visualize the decision boundary of the KNN classifiers using different values for $k = \{1, 5, 9\}$.

2. Use the following dataset:

```
X1, y1 = make_gaussian_quantiles(cov=7.,  
                                n_samples=250, n_features=2,  
                                n_classes=3, random_state=1),
```

in order to apply the nonlinear SVM classification. Use the polynomial kernel and set the `degree`, `coef0`, and `C` parameters to 3, 10, and 5 respectively. Plot the decision boundary of the nonlinear SVM classification.

3. Apply linear SVM regression to the following dataset:

```
X = 6 * np.random.rand(n, 1)  
y = (1 - 2 * X + np.random.randn(n, 1)).ravel().
```

Set the width of the margin to 0.8 and plot the fit line.